Київський національний університет імені Тараса Шевченка

Факультет комп'ютерних наук та кібернетики

ЗВІТ

з лабораторної роботи №3

з дисципліни «Навчання з підкріпленням»

на тему «Глибоке Q-навчання

в задачі CarRacing»

Студента 2 курсу магістратури

групи ММШІ-2

Самуся Дем'яна Михайловича

Київ 2026

1. Мета роботи

Покращити базовий агент DQN (Deep Q-Network) для середовища `CarRacing-v3` з дискретними діями. Налаштування середовища (функція `make_env`) і тестовий скрипт `test_car_racing.py` змінювати не можна, тому покращення стосуються лише навчання: збирання досвіду, цільових значень, гіперпараметрів і вибору контрольної точки. Окрім підсумкового балу, увагу приділено вартості навчання (кількість кадрів середовища і час) і стабільності стратегії на різних трасах.

2. Постановка задачі

У `CarRacing-v3` агент керує автомобілем на випадково згенерованій трасі, яку видно згори. Спостереження – кольоровий кадр 96 × 96. `make_env` переводить його у відтінки сірого, зменшує до 84 × 84 і складає 4 останні кадри в один стан, щоб мережа бачила рух. Дій п'ять: нічого не робити, кермо ліворуч або праворуч (±0,6), газ (0,2) і гальмо (0,8). Кермо не додає газу, тому на поворотах автомобіль їде накатом.

Траса складається з $N \approx 250...330$ плиток. За кожну нову плитку агент отримує $1000/N$, за кожен кадр – $-0{,}1$, за виїзд за межі поля – -100 (епізод завершується). Епізод обмежено 1000 кадрами; коло вважається пройденим, коли автомобіль повертається на старт, відвідавши щонайменше 95 % плиток. Отже, бал епізоду приблизно дорівнює $1000 \cdot f - 0.1 \cdot T$, де f – частка відвіданих плиток, T – кількість кадрів. Добрий агент має не лише триматися траси, а й їхати швидко: якщо за 1000 кадрів коло не пройдено, бал обмежено частиною плиток.

Базовий агент (репозиторій курсу, `dqn_car_racing.py`) – класичний DQN: згорткова мережа Nature DQN (три згорткові шари і два повнозв'язні, 5 виходів $Q(s, a)$), буфер відтворення, цільова мережа і ε-жадібне дослідження. Параметри наведено в табл. 1.

Параметр	**Значення**
навчання	1000 епізодів по 1000 кадрів ($\approx$ 1 млн кадрів), 8 паралельних середовищ
оновлення	2 на крок 8 середовищ (0,25 на кадр), батч 128, Adam, $\mathrm{lr} = 10^{-4}$
ціль	$r + \gamma \max_a Q_{\text{ціл}(s',a)}$, $\gamma = 0.99$, функція втрат Губера
цільова мережа	синхронізація кожні 1000 оновлень
буфер	50 000 переходів
дослідження	ε від 1,0, множення на 0,995 за епізод, нижня межа 0,05

Таблиця 1. Параметри базового агента

Тестовий скрипт проганяє 50 епізодів жадібною стратегією на випадкових трасах, діючи на кожному кадрі. Для базової контрольної точки в умові наведено середній бал 840,6 (стандартне відхилення 103,2, мінімум 470,4, максимум 912,7).

3. Хід роботи

Код роботи розміщено в репозиторії github.com/Kentoso/rl-labs/tree/main/lab3. `dqn_car_racing.py` – базовий скрипт, у якому проводилися перші експерименти, `dqn_car_racing_improved.py` – підсумковий скрипт: базовий агент плюс лише ті зміни, корисність яких підтвердили запуски. Кожну зміну, відхилену ідею й результат кожного запуску з числами записано в `CHANGELOG.md`. Нижче робота викладена етапами в тому порядку, в якому вона велася: кожен етап починається з питання, яке поставив попередній, і закінчується рішенням. Номери запусків збігаються з журналом змін.

3.1. Етап 0. Базова модель і вимірювання

Як виміряти, чи стала модель кращою?

Базовий код і контрольну точку скопійовано з репозиторію курсу без змін; додано лише підтримку GPU Apple (MPS). Мірило результату задане умовою – `test_car_racing.py`: 50 епізодів жадібною стратегією, по одному рішенню на кадр. Траси тесту не фіксовані, тому навіть для тієї самої моделі середній бал є випадковою величиною: зі стандартним відхиленням 100–140 стандартна похибка середнього за 50 епізодів становить 14–19. Отже, різниця в 20–30

балів між двома тестами сама по собі нічого не доводить. Як це вплинуло на висновки, видно на етапах 3, 6 і 8.

Тому кожен запуск з самого початку зберігає детальні журнали: налаштування, кожен навчальний епізод (бал, причина завершення, частка відвіданих плиток, швидкість, оцінка Q першого стану проти фактичної дисконтованої винагороди, частоти дій), кожні 500 оновлень (функція втрат, Q, похибка TD, пропускна здатність, час кожної фази циклу) і періодичні оцінки жадібної стратегії на фіксованих трасах. Аналіз нижче спирається на ці журнали. Тренування проводилося на Google Colab (GPU NVIDIA L4), короткі перевірки – на Apple M3 Max.

Рішення: порівнювати моделі за тестом, а під час навчання – на фіксованих трасах, однакових для всіх запусків, щоб різниці можна було рахувати попарно, трасу з трасою.

3.2. Етап 1. Швидкий цикл навчання

Чи можна навчати достатньо швидко, щоб дозволити собі десяток запусків?

Профілювання показало, що крок циклу (8 середовищ, 2 оновлення) займав 34,3 мс, причому два оновлення коштували більше, ніж крок середовищ (близько 8 мс). Зміни (табл. 2):

- буфер відтворення перенесено на GPU (кадри як `uint8`); батч вибирається індексуванням на пристрої замість копіювання з `numpy`;
- середовища крокують асинхронно (`step_async` / `step_wait`), поки GPU виконує оновлення;
- злитий оптимізатор Adam (`fused=True`) і власне обрізання градієнта однією операцією замість двох десятків дрібних.

Вимір	**до**	**після**
крок циклу, 8 середовищ, MPS	34,3 мс	26,0 мс
оновлення `replay()`, батч 128, MPS	10,1 мс	8,1 мс
оновлення, батч 256, L4 (етап 5: без змішаної точності)	9,4 мс	5,7 мс

Таблиця 2. Вартість циклу навчання

Відхилено після вимірювань: змішану точність на MPS (fp16 повільніший за fp32, bf16 швидший лише на 6 %), зберігання кадрів як float16,

об'єднання двох проходів онлайн-мережі в Double DQN і `torch.compile` – усі давали виграш у межах похибки або програш. Діагностика показала, що обрізання градієнта не спрацьовує (норма до 0,3 при порозі 10).

На цьому ж етапі, ще без перевірки, до агента додано дві зміни з теоретичних міркувань: ціль Double DQN, $r + \gamma Q_{\text{ціл}(s', \arg\max_a Q(s',a))}$, проти переоцінки Q, і масштаб винагороди 0,1 у буфері, щоб цілі були ближчими до початкових виходів мережі. Їх перевірено на етапі 5.

Рішення: залишити буфер на GPU, асинхронні середовища і злитий Adam; обрізання градієнта прибрати пізніше, коли воно не спрацювало в жодному запуску.

3.3. Етап 2. Перший повний запуск і діагноз

Що заважає агенту?

Запуск 1 (Colab, 12 середовищ, $\approx$ 1 млн кадрів – бюджет базового агента, 51 хв) дав на тесті лише 577,0 ± 92,8, і середній бал навчальних епізодів ще зростав. За той самий бюджет базовий результат не відтворився; причину окремо не встановлено.

Журнали показали:

- усі епізоди, крім трьох, закінчились на ліміті 1000 кадрів, агент проходив близько 60 % траси і жодного разу не пройшов коло. Газ у середовищі слабкий (0,2), а агент ще в 15 % кадрів нічого не робив;
- оцінка Q першого стану (40,4) близька до фактичної дисконтованої винагороди (44,3): переоцінки немає;
- похибки TD малі, обрізання градієнта не спрацювало жодного разу;
- цикл обмежений GPU: 9,4 мс на оновлення.

Рішення: спершу зробити окремий експеримент значно дешевшим, бо 51 хв на запуск, що не зійшовся, не дозволяють перевіряти ідеї.

3.4. Етап 3. Швидші експерименти

Де навчання витрачає кадри даремно?

Три зміни, кожна з яких вимикається параметром:

- **обрізання без прогресу** (no-progress truncation): навчальний епізод обривається після 100 кадрів без нової плитки. Нову плитку видно з винагороди: лише вона дає $r > 0$ ($1000/N - 0.1$), усі інші кадри дають −0,1 або

−100. Поріг 100 обрано за вимірюванням: агент, що їде трасою з газом, отримує нову плитку щонайменше кожні 12 кадрів, а випадкова стратегія – з проміжками до 114–139 кадрів. Оцінювання і тест не змінюються;

- **обрив – не кінець:** у запуску 1 ліміт часу (99,7 % епізодів) обнуляв цільове значення, тобто вчив агента, що «світ закінчується» у стані, який нічим не відрізняється від інших. Тепер ліміт часу і обрізання без прогресу продовжують бутстреп $Q(s')$; обнуляє його лише справжнє завершення (виїзд за поле, пройдене коло);
- ε спадає лінійно за кадрами (1,0 → 0,05 за 150 тис.), а бюджет і оцінювання рахуються в кадрах: з обриванням довжина епізоду більше не міряє обсяг навчання.

Заодно змінено розклад оновлень: батч 256 замість 128 при вдвічі меншій кількості оновлень на перехід (0,125 замість 0,25), а цільова мережа синхронізується кожні 500 оновлень замість 1000. Кількість вибраних з буфера переходів на один збережений ($0.125 \cdot 256 = 0.25 \cdot 128 = 32$) і інтервал синхронізації в переходах (4000) ті самі, що в базового агента, тож повторне використання досвіду не змінилось; змінилось лише число кроків оптимізатора – вдвічі менше, з менш шумним градієнтом. Окремо ця зміна не перевірялася.

Запуск 2 (300 тис. кадрів, 13,5 хв) порівняно із запуском 1 на тих самих кадрах наведено в табл. 3.

Запуск	**50 тис.**	**100 тис.**	**150 тис.**	**200 тис.**	**250 тис.**	**300 тис.**
1 (базовий цикл)	−90	−8	−54	−60	−45	−27
2 (етап 3)	−79	−51	−16	−17	312	**359**

Таблиця 3. Середній бал на 10 трасах при однаковій кількості кадрів

Запуск 2 на 300 тис. кадрів досяг рівня, якого запуск 1 досяг лише на 550–650 тис., тобто потребує приблизно вдвічі менше даних. Обрізання без прогресу перестає діяти, щойно агент починає їхати: середня довжина епізоду виросла з 281 до 951 кадру.

Запуск 3 (ті самі налаштування, 1,5 млн кадрів, 68,5 хв) дав першу модель, близьку до базової: контрольна точка на 1,05 млн кадрів отримала на тесті 829,3 ± 86,2. Але `best.pth` того самого запуску – точка з найвищим балом

на 10 трасах оцінювання (789 на 1,2 млн) – на тесті отримала лише 753,7 ± 198,5 з мінімумом −76. На 10 трасах зі стандартним відхиленням 100–200 похибка середнього становить 30–60, тобто стільки ж, скільки різниця між сусідніми контрольними точками. Тому найвищий бал на 10 трасах здебільшого означав вдалий для цієї точки набір трас, а не кращу модель.

Журнали запуску 3 уточнили діагноз: на ліміті часу агент відвідував близько 87 % плиток, коло проходив лише на одній трасі з десяти, а бал $\approx 1000 \cdot f - 100$. Похибки TD малі, оцінки Q збігаються з фактичною винагородою, тож мережа не схожа на вузьке місце. Агент просто їде надто повільно.

Рішення: обрізання без прогресу, бутстреп на обриванні й лінійний ε залишити; оцінювати на 30 трасах замість 10; наступна мета – навчити агента їхати швидше.

3.5. Етап 4. Повторення дії

Чи допоможе агенту приймати рішення рідше?

Повторення дії (action repeat, frame skip) під час навчання: кожне рішення агента утримується k кадрів, винагороди за ці кадри сумуються, а цільове значення дисконтується на γ^k. Стан, як і раніше, – 4 останні кадри з `make_env`, тому стратегію можна застосовувати на кожному кадрі; оцінювання і тест так і роблять, тож кожне оцінювання перевіряє цей перенос. Очікування: одне рішення охоплює вчетверо більше часу, тому винагорода поширюється назад за вчетверо менше кроків бутстрепу ($\gamma^4 \approx 0.96$ на рішення), а випадкові дії на початку навчання не скасовують одна одну щокадрово. Разом з повторенням дії додано dueling-архітектуру (Wang et al., 2016): $Q(s,a) = V(s) + A(s,a) - \overline{A}(s)$, бо на прямих ділянках більшість дій майже рівноцінні.

Два короткі запуски по 100 тис. кадрів на ноутбуці з однаковою кількістю оновлень на кадр (табл. 4):

Запуск	**25 тис.**	**50 тис.**	**75 тис.**	**100 тис.**
4: $k = 4$	197	391	658	**750**
5: $k = 1$	−64	−85	−78	9

Таблиця 4. Повторення дії: середній бал на 10 трасах

З $k = 4$ агент за 100 тис. кадрів і 9 хв на ноутбуці отримав на тесті 780,8 ± 92,1 з мінімумом 562,6 – вищим, ніж у будь-якої точки запуску 3, якому знадобилося 1,05 млн кадрів і 50 хв на L4. Без повторення агент за той самий час ледве рушає з місця.

Рішення: $k = 4$ залишити. Dueling-архітектура, Double DQN і масштаб винагороди досі не перевірені окремо – це наступний етап.

3.6. Етап 5. Мінімальний скрипт і перевірка складових

Що з доданого справді потрібне?

Підсумковий скрипт `dqn_car_racing_improved.py` скопійовано з базового і перенесено в нього лише підтверджені зміни: пришвидшення циклу, етап 3, $k = 4$ і оцінювання на 30 трасах. Заодно прибрано змішану точність, яку раніше не вимірювали на L4: без неї оновлення пришвидшилося з 9,4 до 5,7 мс. Три неперевірені зміни – масштаб винагороди, Double DQN і dueling – спершу прибрано, з критерієм, узгодженим заздалегідь: на 100 тис. кадрів результат має бути не більш ніж на 150 нижчим за 750 запуску 4 на тих самих трасах.

Перевірка 1 (без усіх трьох) дала 519 ± 242 – не пройшла. Оцінки Q сильно відставали від фактичної винагороди (15 проти 54).

Перевірка 2 (масштаб 0,1 повернуто) усунула відставання Q (27 проти 38), але дала 453 ± 270 з кількома провальними трасами. Щоб зрозуміти, чому масштаб важливий, на кінцевих моделях обох перевірок виміряно розподіл похибок TD (табл. 5). З повторенням дії один перехід сумує 4 кадри винагороди (до 10–11 одиниць), і без масштабу три чверті похибок потрапляють у лінійну частину функції Губера ($|\delta| \geq 1$), де градієнт обмежено ±1 незалежно від розміру похибки: функція втрат поводиться майже як L1 і тягне Q до медіани цілей. З масштабом вона фактично квадратична.

Модель	**середня $\|\delta\|$**	**медіана**	**p99**	**$\|\delta\| \geq 1$ (лінійна частина)**
без масштабу (одиниці середовища)	2,22	1,90	8,33	**76,5 %**
масштаб 0,1	0,21	0,17	0,78	0,2 %

Таблиця 5. Похибки TD на кінцевих моделях перевірок 1 і 2 (≈ 3000 переходів)

Перевірка 3 повернула Double DQN (635 ± 161, мінімум 370) і водночас повторила еталон запуску 4 з іншим seed: він отримав 652 замість 750. Отже, 750 було вдалим випадком, а розкид між seed на 100 тис. кадрів становить близько 100. Новий скрипт з масштабом і Double DQN у цих межах – перевірку пройдено. Без Double DQN деякі траси провалювались повністю (мінімум 15), з ним – ні.

Рішення: масштаб винагороди 0,1 і Double DQN залишити; dueling-архітектура непотрібна, і мережа лишається такою самою, як у базового агента.

3.7. Етап 6. Плато

Чи дасть довше навчання кращу модель?

Запуск 6 (новий скрипт, 500 тис. кадрів, 38 хв) вийшов на плато близько 750 з 250 тис. кадрів (рис. 2) і отримав на тесті 769,1 ± 136,4. На плато оцінка Q першого стану постійно росла (24 → 57), тоді як фактична дисконтована винагорода залишалась на рівні 40–47 (рис. 1): переоцінка, попри Double DQN.

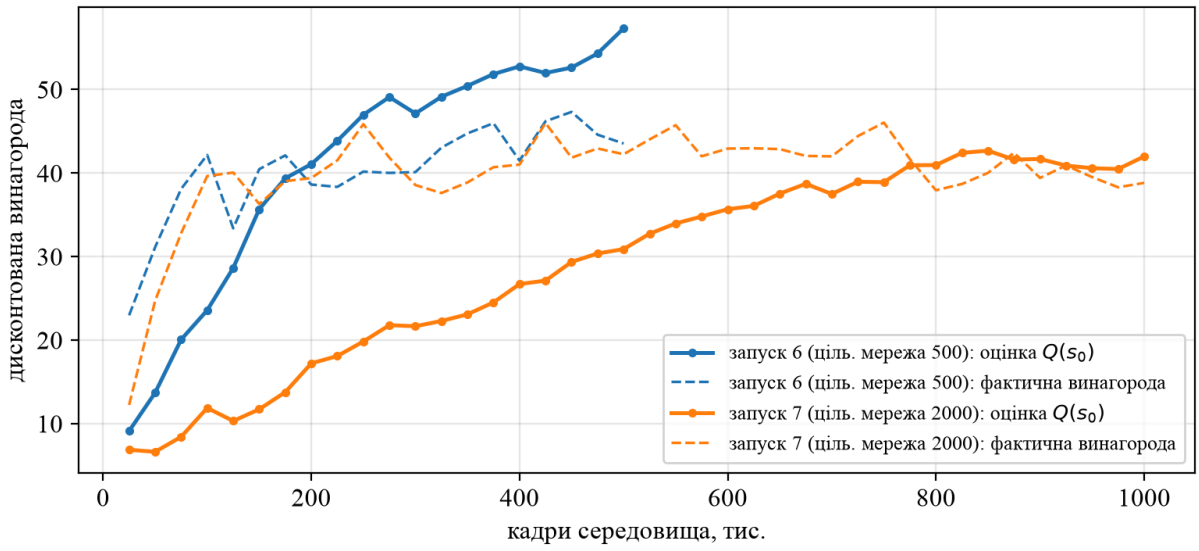


Рис. 1. Оцінка Q першого стану і фактична дисконтована винагорода на 30 трасах оцінювання (одиниці середовища)

Запуск 7 (1 млн кадрів, 77 хв) поєднав три зміни, тож їхній внесок окремо не розділено:

- цільова мережа синхронізується вчетверо рідше (кожні 2000 оновлень) – проти переоцінки;
- k зменшується 4 → 3 → 2 → 1 на 300, 500 і 750 тис. кадрів: грубе керування, поки агент вчиться, і точніше потім, а наприкінці – навчання

саме так, як агент діє на тесті. Щоб переходи з різним k могли лежати в буфері разом, кожен перехід зберігає власний множник $\gamma^{\text{кадрів}}$;

- швидкість навчання спадає лінійно з 300 тис. кадрів до 10^{-5}.

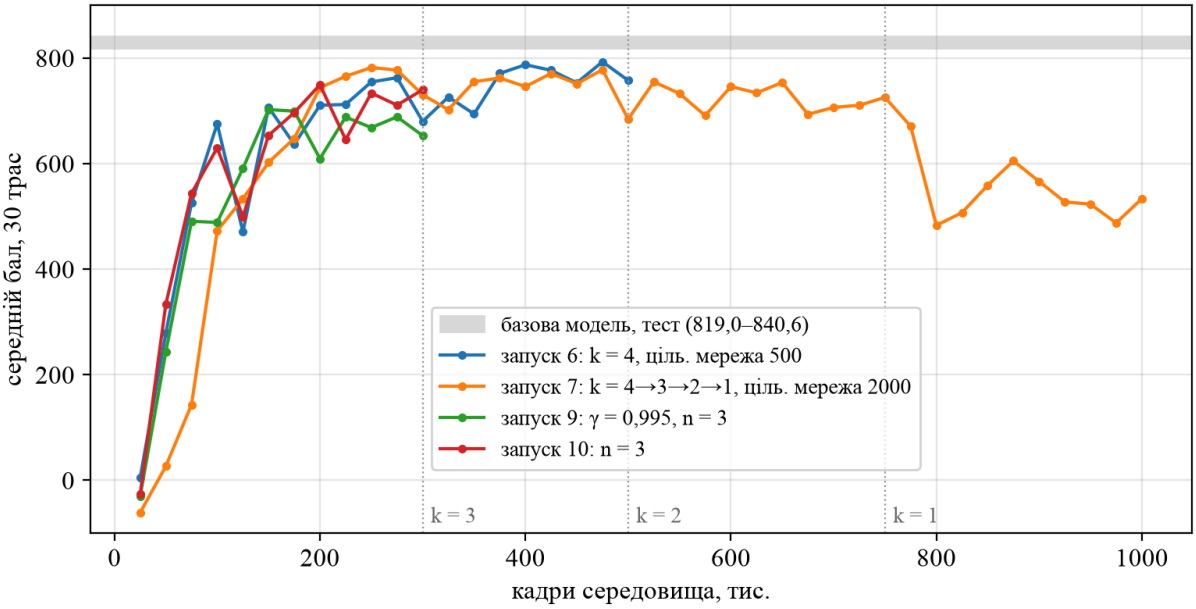


Рис. 2. Середній бал жадібної стратегії на 30 фіксованих трасах під час навчання. Пунктир – зміна k у запуску 7; сіра смуга – базова модель на тесті

Повільніша цільова мережа прибрала переоцінку: Q підходить до фактичної винагороди знизу і зупиняється на ній (рис. 1), ціною повільнішого початку. $k = 3$ і $k = 2$ плато не підняли, а $k = 1$ зіпсував стратегію: з 725 до 483 за 50 тис. кадрів, з повільнішою їздою і знову частими обриваннями без прогресу. Найкраща контрольна точка – з фази $k = 4$ (250 тис. кадрів, $\approx$ 20 хв), тест 798,4 ± 85,3 з мінімумом 619,9.

Тут довелося переглянути попередній висновок. За тестом запуск 6 був на 72 нижчим за 840,6 з умови, і це здавалося надійним програшем. Повторний тест самої базової контрольної точки дав 819,0 ± 136,7: те саме значення умови виявилось вдалим випадком. Проти 819,0 запуск 7 нижчий на 20,6 при стандартній похибці різниці $\approx$ 23 – у межах шуму.

Рішення: поступове зменшення k відкинути; цільову мережу кожні 2000 оновлень залишити.

3.8. Етап 7. Як їхати швидше

Чому агент їде повільно і чи можна це виправити ціллю?

Найкраща оцінка запуску 7 показала, що 24 з 30 трас закінчуються на ліміті часу з 85 % плиток, 6 – пройденим колом, а бал корелює зі швидкістю (0,67). Агент майже не гальмує (0,3 % кадрів) і рідко злітає з траси – він обережний. Перевірено дві ідеї:

- **довший горизонт**, $\gamma = 0.995$ замість 0,99 ($\approx$ 200 кадрів замість 100): щоб агент бачив наслідки, що настають пізніше, – втрачені кадри після заносу, вихід з повороту;
- **n-крокові винагороди** ($n = 3$ рішення, 12 кадрів до бутстрепу): винагорода поширюється назад утричі швидше.

Короткий запуск 8 на ноутбуці (100 тис. кадрів, обидві зміни) дав 729 проти 473 у запуску 7 на тих самих трасах – попарно +257 ± 57. Але запуск 9 з тими самими налаштуваннями і тим самим seed на Colab мав на 100 тис. лише 488: обчислення на MPS і CUDA не збігаються побітово, запуски розходяться, і один запуск на 100 тис. кадрів може відхилятися на 250. Результат запуску 8 був вдалим випадком. Підсумок у табл. 6.

Варіант	**75 тис.**	**100 тис.**	**200 тис.**	**300 тис.**	**тест**
7: $\gamma = 0.99$, $n = 1$	143	473	744	730	**798,4**
9: $\gamma = 0.995$, $n = 3$	491	488	609	653	645,6
10: $\gamma = 0.99$, $n = 3$	544	630	749	740	750,3

Таблиця 6. Середній бал на 30 трасах і тест (одна seed на варіант)

$n = 3$ стабільно пришвидшує початок (на 300–400 вище за запуск 7 на 50–75 тис. кадрів у трьох запусках поспіль), але плато не піднімає. $\gamma = 0.995$ шкодить: оцінки Q стають удвічі більшими, і з повільною цільовою мережею мережа не встигає їх наздогнати (47 проти 88 на 300 тис. кадрів).

Інше пояснення обережності – саме повторення дії: можливо, утримання дії на 4 кадри робить стратегію «надто рішучою». Найкращу точку запуску 7 оцінено на тих самих 30 трасах, утримуючи кожну дію k кадрів уже під час оцінювання (табл. 7).

Рішення кожні	**середнє**	**мінімум**	**плитки**	**різниця з $k = 1$**
1 кадр	749,6	272,1	84,8 %	–
2 кадри	754,7	403,9	85,3 %	+5 ± 20
4 кадри (як у навчанні)	592,9	−87,4	69,3 %	−157 ± 40

Таблиця 7. Модель, навчена з $k = 4$, при різній частоті рішень на оцінюванні

Результат протилежний гіпотезі: стратегія суттєво використовує можливість виправляти курс між своїми 4-кадровими рішеннями, хоча її оцінки Q таких виправлень ніколи не бачили. Рішення кожні 2 кадри не гірші за щокадрові. Побічне спостереження: та сама модель на тих самих трасах отримала 781,7 на L4 і 749,6 на MPS – одне оцінювання на 30 трасах коливається приблизно на ±30 навіть для фіксованої моделі.

Рішення: $\gamma = 0.99$ і $n = 1$ залишити (n-крокові винагороди – як опція для коротких перевірок). Остання гіпотеза: навчання з $k = 2$ відповідає частоті керування, яку стратегія справді використовує.

3.9. Етап 8. Підсумкова модель

Яку модель здавати і чи вона краща за базову?

Спершу перевірено останню гіпотезу етапу 7: запуски 11 і 12 – конфігурація запуску 7 з $k = 2$ замість 4, два seed, по 300 тис. кадрів (рис. 3). Кількість оновлень на кадр та сама.

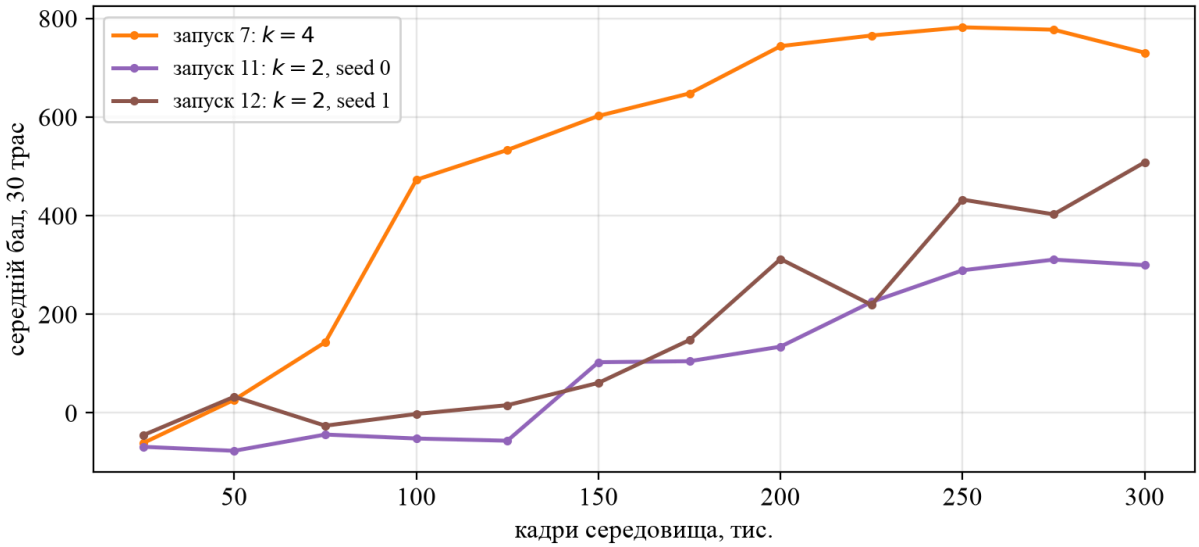


Рис. 3. Навчання з $k = 2$ (два seed) проти $k = 4$ (запуск 7) на тих самих 30 трасах

Обидва seed навчаються значно повільніше: до $\approx$ 150 тис. кадрів бал близький до нуля, тоді як з $k = 4$ на 100 тис. кадрів він уже 473. На 300 тис. кадрів запуски 11 і 12 мають 300 і 508 – приблизно рівень запуску 7 на 100 тис. – і ще ростуть; на кожному оцінюванні з 75 тис. кадрів вони нижчі за запуск 7 на 169–609. Оцінки Q сильно відстають від фактичної винагороди (11–14 проти 30–42): з рішенням кожні 2 кадри ланцюжок бутстрепу до того самого горизонту вдвічі довший. Отже, те, що модель з $k = 4$ використовує виправлення між рішеннями, не означає, що навчатися з $k = 2$ вигідніше: за цього бюджету переважає швидкість навчання. Запуск 3 показує, що й з $k = 1$ можна отримати добру модель, але за $\approx$ 1 млн кадрів. Ці моделі на 100 трасах і тесті не оцінювались – вони на сотні балів нижчі за запуск 7 на тих самих трасах.

Отже, кандидати – точки запусків 3 і 7.

Тест не розрізняє моделі, що відрізняються на 20–30 балів, тому кандидатів і базову модель оцінено на 100 нових трасах (seed 20 000–20 099, не перетинаються з трасами оцінювання). Траси однакові для всіх моделей, тож різниці рахуються попарно, і стандартна похибка різниці з базовою моделлю становить 13–16 (табл. 8).

Модель	**сер.**	**ст. відх.**	**мін.**	**кола**	**різниця з базовою**
базова	797,3	167,4	319,6	**39**	–
запуск 3, 1,05 млн кадрів ($k = 1$)	**821,5**	102,4	355,5	20	**+24,2 ± 12,9**
запуск 7, 425 тис.	791,1	112,2	351,4	13	−6,1 ± 13,9
запуск 7, 375 тис.	784,9	105,6	398,5	13	−12,3 ± 15,6
запуск 7, 250 тис. (`best.pth`)	782,2	117,3	310,1	17	−15,1 ± 14,6
запуск 7, 475 тис.	777,3	106,3	369,1	10	−20,0 ± 14,3

Таблиця 8. 100 нових трас, попарно з базовою моделлю

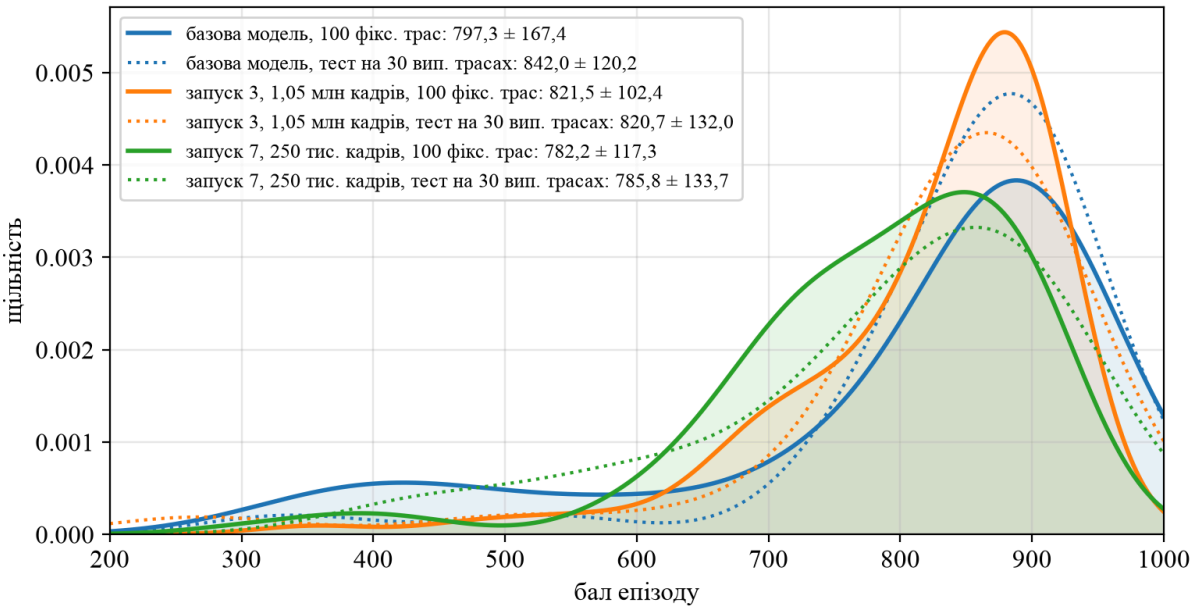


Рис. 4. Розподіл балів (ядрова оцінка щільності, гауссове ядро, ширина за правилом Сільвермана). Суцільні лінії – 100 фіксованих трас, однакових для всіх моделей; пунктирні – `test_car_racing.py` на 30 випадкових трасах, різних для кожної моделі. Найвищі бали епізодів – близько 910–920; щільність правіше від них – результат згладжування

- Точки запуску 7 від 250 до 475 тис. кадрів практично однакові (≈ 780–790): на плато стратегія не змінюється, і вибір за 30 трасами був правильним. З базовою моделлю вони в межах однієї стандартної похибки.
- **Базова модель швидка, але крихка; наші – стабільні, але повільні.** Базова проходить коло на 39 трасах зі 100 проти 10–20 і краща на 70–76 трасах зі 100, але там, де помиляється, помиляється сильно (стандартне відхилення 167 проти 102–117). Наші моделі вигравають за середнім лише завдяки тому, що уникають цих провалів. Це видно на рис. 4: у базової моделі пік правіше (пройдені кола, 880–900) і довгий хвіст провалів до 300–500, у запуску 3 пік вужчий і хвіст тонший, а запуск 7 зміщений ліворуч – повільніше, з тоншим за базову модель хвостом провалів. Офіційний тест на 30 випадкових трасах (пунктир) дає схожі за формою розподіли, але середні знову залежать від трас: базова модель отримала 842,0 ± 120,2 проти 797,3 на фіксованих трасах, запуск 3 – 820,7 ± 132,0, запуск 7 – 785,8 ± 133,7. Зі стандартною похибкою ≈ 22–24 на 30 епізодів і різними трасами для кожної моделі такий тест не розрізняє моделі, що відрізняються на 20–40 балів; тому підсумкове порівняння – попарне на 100 фіксованих трасах.

- Єдиний кандидат попереду – точка запуску 3: +24 ± 13 (≈ 1,9 стандартної похибки), тобто ймовірна, але не доведена перевага.

Рішення: результатів два, і кожен – найкращий за своїм критерієм:

- **за балом** – точка запуску 3 на 1,05 млн кадрів: +24,2 ± 12,9 попарно з базовою моделлю на 100 трасах, тест 829,3 ± 86,2 з мінімумом 420,8. Її здано як підсумкову модель (`lab3/checkpoint_CarRacing-v3_run3_score.pth`);
- **за вартістю навчання** – точка запуску 7 на 250 тис. кадрів: рівна базовій за середнім (−15,1 ± 14,6) і стабільніша за неї, але отримана за 19 хв навчання на L4 і вчетверо менший обсяг даних (`lab3/checkpoint_CarRacing-v3_run7_fast.pth`).

3.10. Підсумкова таблиця запусків

№	етап	що змінено	кадрів, тис.	оцін.	тест	100 трас
–	0	базова модель	1000	–	840,6 / 819,0	–
1	2	Double DQN, масштаб 0,1, Colab	1000	541	577,0	–
2	3	обрізання без прогресу, бутстреп на обриві, ε за кадрами	300	359	–	–
3	3	те саме, довше	1050	766	**829,3**	**+24,2 ± 12,9**
4	4	$k = 4$, dueling	100	750	780,8	–
5	4	$k = 1$, dueling	100	9	–	–
п1–3	5	мінімальний скрипт без / з масштабом і Double DQN	100	519–635	–	–
6	6	мінімальний скрипт, $k = 4$	475	793	769,1	–
7	6	ціль. мережа 2000, розклад k, спад lr	250	782	798,4	−15,1 ± 14,6
8	7	$\gamma = 0.995$, $n = 3$ (MPS)	100	729	–	–
9	7	$\gamma = 0.995$, $n = 3$	150	702	645,6	–
10	7	$n = 3$	200	749	750,3	–
11–12	8	$k = 2$, два seed	275 / 300	311 / 508	–	–

Таблиця 9. Усі запуски. Для кожного – контрольна точка, що згадується в тексті: «кадрів» – тисяч кадрів навчання до неї, «оцін.» – її середній бал на трасах оцінювання (10 трас до етапу 5, далі 30), «тест» – 50 епізодів, «100 трас» – різниця з базовою моделлю попарно

4. Висновки

Без змін середовища і тестового скрипта отримано два результати (табл. 10):

- **найкращий бал** – модель запуску 3 (повна кількість рішень, $k = 1$, 1,05 млн кадрів): на 100 однакових трасах вона на 24 ± 13 краща за базову, зі значно меншим розкидом (стандартне відхилення 102 проти 167);
- **найшвидше навчання** – модель запуску 7 (повторення дії, $k = 4$, 250 тис. кадрів): рівна базовій за середнім, але навчена за 19 хв замість понад години.

Навчання стало значно дешевшим. Навчання за бюджетом базового агента ($\approx$ 1 млн кадрів) на GPU L4 займало 51 хв, причому вже з частиною пришвидшень етапу 1, і дало лише 577 балів; першу модель рівня базової (запуск 3) отримано після 1,05 млн кадрів і $\approx$ 50 хв, а весь запуск тривав 68,5 хв. Модель запуску 7 рівна базовій за середнім на 100 однакових трасах після 250 тис. кадрів і 19 хв навчання. Тобто час навчання скоротився з понад години (або з $\approx$ 50 хв до першої моделі рівня базової) до $\approx$ 20 хв – у 2,5–3,5 раза – за майже того самого результату.

Запуск	**кадрів**	**хв**	**тест**	**100 трас, різниця з базовою**
1: цикл базового агента + Double DQN, масштаб 0,1	1 млн	51	577,0	–
3: + етап 3, $k = 1$	1,05 млн	$\approx$ 50	829,3	+24,2 ± 12,9
7: підсумковий скрипт, $k = 4$	250 тис.	**19**	798,4	−15,1 ± 14,6

Таблиця 10. Час навчання до моделі (Colab, GPU L4; разом з оцінюваннями під час навчання)

Звідки цей виграш: найбільший внесок дали повторення дії на 4 кадри під час навчання (750 проти 9 балів на 100 тис. кадрів) і обрізання навчальних епізодів без прогресу з бутстрепом на обриванні (удвічі менше кадрів для того самого рівня). Для повторення дії виявились необхідними масштаб винагороди 0,1 (інакше три чверті похибок TD потрапляють у лінійну частину функції Губера) і Double DQN; повільніша цільова мережа прибрала переоцінку Q на плато. Інженерні зміни (буфер на GPU, асинхронні середовища, без змішаної

точності) пришвидшили оновлення з 9,4 до 5,7 мс, що й дозволило провести понад десять запусків.

Перевага запуску 3 ($\approx 1{,}9$ стандартної похибки, до того ж найкращий із семи кандидатів) ймовірна, але не доведена. Запуск 7 рівний базовій за середнім (-15 ± 15) при вчетверо меншому обсязі даних. Обидві моделі стабільніші за базову: стандартне відхилення 102–117 проти 167. Натомість базова модель швидша: вона проходить коло на 39 трасах зі 100 проти 13–20.

Швидкість і залишилась головним обмеженням: більшість трас закінчуються на ліміті 1000 кадрів з $\approx$ 85 % плиток. Довший горизонт ($\gamma = 0.995$) і n-крокові винагороди цього не виправили: перший шкодить, другі лише пришвидшують початок. Поступове зменшення k до 1 зіпсувало стратегію. Модель, навчена з $k = 4$, суттєво виграє від частіших рішень, ніж бачила під час навчання, але навчання з $k = 2$ за 300 тис. кадрів відстає від $k = 4$ на сотні балів: частіші рішення потребують значно більше даних, як і в запуску 3 з $k = 1$.

Методичний висновок: шум вимірювань виявився того самого порядку, що й ефекти, які перевірялися. Тест тієї самої моделі змінюється на ≈ 20, оцінювання на 30 трасах – на ≈ 30 між GPU, а запуски з різним seed – на ≈ 100. Тричі висновок за одним вимірюванням довелося переглянути (вибір точки за 10 трасами, «надійний» програш базовій моделі, вдалий запуск 8), тому підсумкові порівняння зроблено попарно на 100 однакових трасах.

Код, журнал змін і обидві підсумкові контрольні точки доступні в репозиторії:

github.com/Kentoso/rl-labs/tree/main/lab3